Week 5: Velocity Kinematics & The Jacobian

EE0849 Introduction to Robotics - Detailed Notes

Basri Erdogan

Table of contents

1 Introduction to Velocity Kinematics

1.1 Motivation

In the previous weeks, we studied **position kinematics**:

- **Forward Kinematics (FK):** Given joint angles $\mathbf{q}$, find end-effector pose $\mathbf{x}$
- **Inverse Kinematics (IK):** Given end-effector pose $\mathbf{x}$, find joint angles $\mathbf{q}$

Now we study **velocity kinematics**, which answers:

- How do joint velocities $\dot{\mathbf{q}}$ relate to end-effector velocity $\mathbf{v}$?
- This relationship is captured by the **Jacobian matrix**

1.2 Why Study Velocity Kinematics?

1. **Real-time control:** Trajectory following requires computing joint velocities
2. **Singularity analysis:** The Jacobian reveals configurations where mobility is lost
3. **Force/torque mapping:** The Jacobian transpose maps end-effector forces to joint torques
4. **Manipulability:** Quantify how well the robot can move at any configuration
5. **Redundancy resolution:** For robots with extra DOF, choose among infinite IK solutions

1.3 The Fundamental Velocity Equation

The Jacobian J linearly relates joint velocities to end-effector velocity:

$$\mathbf{v} = J(\mathbf{q}) \cdot \dot{\mathbf{q}}$$

Where:

- $\mathbf{v} \in \mathbb{R}^m$: End-effector velocity (task space)
- $\dot{\mathbf{q}} \in \mathbb{R}^n$: Joint velocity vector
- $J(\mathbf{q}) \in \mathbb{R}^{m \times n}$: Jacobian matrix

Key insight: The Jacobian is configuration-dependent. It must be recomputed at each robot pose.

2 Deriving the Jacobian

2.1 Analytical Approach: Differentiation

For forward kinematics $\mathbf{x} = f(\mathbf{q})$, the Jacobian is the matrix of partial derivatives:

$$J_{ij} = \frac{\partial x_i}{\partial q_j}$$

Or in matrix form:

$$J = \frac{\partial \mathbf{x}}{\partial \mathbf{q}} = \begin{bmatrix} \frac{\partial x_1}{\partial q_1} & \frac{\partial x_1}{\partial q_2} & \cdots & \frac{\partial x_1}{\partial q_n} \\ \frac{\partial x_2}{\partial q_1} & \frac{\partial x_2}{\partial q_2} & \cdots & \frac{\partial x_2}{\partial q_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial x_m}{\partial q_1} & \frac{\partial x_m}{\partial q_2} & \cdots & \frac{\partial x_m}{\partial q_n} \end{bmatrix}$$

2.2 Example: 2-Link Planar Arm

From forward kinematics:

$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2)$$
$$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2)$$

Taking partial derivatives:

$$\frac{\partial x}{\partial \theta_1} = -L_1 \sin\theta_1 - L_2 \sin(\theta_1 + \theta_2)$$
$$\frac{\partial x}{\partial \theta_2} = -L_2 \sin(\theta_1 + \theta_2)$$
$$\frac{\partial y}{\partial \theta_1} = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2)$$
$$\frac{\partial y}{\partial \theta_2} = L_2 \cos(\theta_1 + \theta_2)$$

2.3 The 2-Link Jacobian

Assembling the derivatives:

$$J = \begin{bmatrix} -L_1 \sin\theta_1 - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

Using shorthand notation ($s_1 = \sin\theta_1$, $c_1 = \cos\theta_1$, $s_{12} = \sin(\theta_1 + \theta_2)$, $c_{12} = \cos(\theta_1 + \theta_2)$):

$$J = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}$$

2.4 Python Implementation

```python
import numpy as np

def jacobian_2link(theta1, theta2, L1, L2):
    """
    Compute the 2x2 Jacobian for a 2-link planar arm.

    Parameters:
        theta1, theta2: Joint angles (radians)
        L1, L2: Link lengths (meters)

    Returns:
        J: 2x2 Jacobian matrix
    """
    s1 = np.sin(theta1)
    c1 = np.cos(theta1)
    s12 = np.sin(theta1 + theta2)
    c12 = np.cos(theta1 + theta2)

    J = np.array([
        [-L1*s1 - L2*s12, -L2*s12],
        [ L1*c1 + L2*c12,  L2*c12]
    ])
    return J
```

3 Jacobian Column Interpretation

3.1 Physical Meaning of Columns

The Jacobian can be written in terms of its columns:

$$J = \begin{bmatrix} \mathbf{j}_1 & \mathbf{j}_2 & \cdots & \mathbf{j}_n \end{bmatrix}$$

Each column $\mathbf{j}_i$ represents the end-effector velocity when:

- Joint i moves at unit velocity ($\dot{q}_i = 1$)
- All other joints are stationary ($\dot{q}_j = 0$ for $j \neq i$)

3.2 Geometric Construction for 3D Robots

For a 3D robot, the full Jacobian has 6 rows (3 linear + 3 angular velocity):

$$J = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix}$$

For a revolute joint i:

$$\mathbf{j}_i = \begin{bmatrix} \mathbf{z}_{i-1} \times (\mathbf{p}_n - \mathbf{p}_{i-1}) \\ \mathbf{z}_{i-1} \end{bmatrix}$$

Where:

- $\mathbf{z}_{i-1}$: Unit vector along joint axis
- $\mathbf{p}_n$: End-effector position

- $\mathbf{p}_{i-1}$: Position of joint $i-1$

For a prismatic joint i:

$$\mathbf{j}_i = \begin{bmatrix} \mathbf{z}_{i-1} \\ \mathbf{0} \end{bmatrix}$$

4 Singularities

4.1 Definition

A **singularity** is a configuration where the Jacobian loses rank:

$$\text{rank}(J) < \min(m, n)$$

For a square Jacobian ($m = n$), this occurs when:

$$\det(J) = 0$$

4.2 Physical Interpretation

At a singularity:

1. **Loss of mobility:** The robot cannot move in certain directions
2. **Infinite joint velocities:** Achieving finite task velocity requires infinite joint velocities
3. **Ambiguous inverse:** The inverse velocity problem has no unique solution

4.3 2-Link Arm Singularities

Computing the determinant:

$$\det(J) = (-L_1 s_1 - L_2 s_{12})(L_2 c_{12}) - (-L_2 s_{12})(L_1 c_1 + L_2 c_{12})$$

After simplification:

$$\det(J) = L_1 L_2 \sin\theta_2$$

Singularities occur when $\sin\theta_2 = 0$**, i.e.,** $\theta_2 = 0$ **or** $\theta_2 = \pm\pi$

These correspond to:

- $\theta_2 = 0$: Arm fully extended (at workspace boundary)
- $\theta_2 = \pm\pi$: Arm fully folded (at workspace boundary)

4.4 Types of Singularities

Type	Description	Example
Boundary	At workspace edge	Fully extended arm
Interior	Inside workspace	Wrist axes aligned
Algorithmic	Due to representation	Gimbal lock

4.5 Python Singularity Detection

```python
def is_singular(J, threshold=1e-6):
    """Check if Jacobian is singular."""
    if J.shape[0] == J.shape[1]:
        return abs(np.linalg.det(J)) < threshold
    else:
        # For non-square, check smallest singular value
        S = np.linalg.svd(J, compute_uv=False)
        return S[-1] < threshold

def singularity_measure(theta2, L1, L2):
    """Compute singularity measure (determinant) for 2-link arm."""
    return L1 * L2 * np.sin(theta2)
```

5 Inverse Velocity Kinematics

5.1 The Inverse Problem

Given a desired end-effector velocity $\mathbf{v}_{des}$, find joint velocities:

$$\dot{\mathbf{q}} = J^{-1}\mathbf{v}_{des}$$

This requires inverting the Jacobian.

5.2 Conditions for Invertibility

The Jacobian is invertible when:

1. It is **square** ($m = n$)
2. It has **full rank** (not at a singularity)

5.3 Non-Square Jacobians

Case	Condition	Solution
Redundant	$n > m$	Infinite solutions
Under-actuated	$n < m$	Generally no solution

5.4 Pseudoinverse Solution

For non-square or rank-deficient Jacobians, use the **Moore-Penrose pseudoinverse**:

$$\dot{\mathbf{q}} = J^{+}\mathbf{v}_{des}$$

Right pseudoinverse (for $n > m$, redundant case):

$$J^{+} = J^{T}(JJ^{T})^{-1}$$

This gives the **minimum-norm** solution: smallest $\|\dot{\mathbf{q}}\|$ that achieves $\mathbf{v}_{des}$.

Left pseudoinverse (for $n < m$, under-actuated case):

$$J^{+} = (J^{T}J)^{-1}J^{T}$$

This gives the **least-squares** solution: minimizes $\|J\dot{\mathbf{q}} - \mathbf{v}_{des}\|$.

5.5 Damped Least Squares (DLS)

Near singularities, the inverse becomes numerically unstable. Use **damped least squares**:

$$\dot{\mathbf{q}} = J^T(JJ^T + \lambda^2 I)^{-1}\mathbf{v}_{des}$$

- λ: Damping factor (tuning parameter)
- Larger $\lambda \rightarrow$ more damping, less accuracy
- Smaller $\lambda \rightarrow$ less damping, may be unstable near singularities

```
def inverse_jacobian_dls(J, v_desired, damping=0.01):
    """Compute joint velocities using damped least squares."""
    JJT = J @ J.T
    n = JJT.shape[0]
    return J.T @ np.linalg.inv(JJT + damping**2 * np.eye(n)) @ v_desired
```

6 Resolved-Rate Motion Control

6.1 Concept

Use the Jacobian for real-time trajectory following:

1. Compute position error: $\mathbf{e} = \mathbf{x}_{des} - \mathbf{x}_{cur}$
2. Generate velocity command: $\mathbf{v} = K \cdot \mathbf{e}$
3. Compute joint velocities: $\dot{\mathbf{q}} = J^{-1}\mathbf{v}$
4. Integrate: $\mathbf{q}_{new} = \mathbf{q} + \dot{\mathbf{q}} \cdot \Delta t$
5. Repeat

6.2 Algorithm

```
def resolved_rate_step(q, x_desired, L1, L2, K=1.0, dt=0.01, damping=0.01):
    """
    One step of resolved-rate motion control.

    Parameters:
        q: Current joint angles [theta1, theta2]
        x_desired: Desired end-effector position [x, y]
        L1, L2: Link lengths
        K: Proportional gain
        dt: Time step
        damping: DLS damping factor

    Returns:
        q_new: Updated joint angles
    """
    # Current end-effector position
    x_current = fk_2link(q[0], q[1], L1, L2)

    # Position error
    error = np.array(x_desired) - np.array(x_current)

    # Desired velocity (proportional control)
```

```
    v_desired = K * error

    # Jacobian
    J = jacobian_2link(q[0], q[1], L1, L2)

    # Joint velocities (using DLS for robustness)
    q_dot = inverse_jacobian_dls(J, v_desired, damping)

    # Integrate
    q_new = q + q_dot * dt

    return q_new
```

6.3 Convergence and Stability

- **Gain K:** Too high $\rightarrow$ oscillations, too low $\rightarrow$ slow convergence
- **Damping λ:** Higher near singularities for stability
- **Time step Δt:** Smaller for accuracy, larger for speed

7 Manipulability

7.1 Definition

Manipulability measures how well the robot can move at a given configuration:

$$w = \sqrt{\det(JJ^T)}$$

For a square Jacobian:

$$w = |\det(J)|$$

Properties:

- $w = 0$ at singularities
- Higher $w \rightarrow$ better mobility in all directions
- w is always non-negative

7.2 Manipulability Ellipsoid

The set of achievable end-effector velocities for bounded joint velocities:

$$\|\dot{\mathbf{q}}\| \leq 1 \quad \Rightarrow \quad \mathbf{v}^T(JJ^T)^{-1}\mathbf{v} \leq 1$$

This defines an **ellipsoid** in velocity space.

7.3 Ellipsoid Properties

Using SVD of $J = U\Sigma V^T$:

- **Principal axes:** Columns of U
- **Axis lengths:** Singular values $\sigma_1, \sigma_2, \ldots$
- **Volume:** Proportional to $\prod_i \sigma_i = w$

```python
def manipulability_ellipse(J):
    """Compute manipulability ellipse parameters."""
    U, S, Vt = np.linalg.svd(J)

    # Principal directions (columns of U)
    directions = U

    # Axis lengths (singular values)
    lengths = S

    # Manipulability measure
    w = np.prod(S)

    # Condition number
    kappa = S[0] / S[-1] if S[-1] > 1e-10 else np.inf

    return directions, lengths, w, kappa
```

7.4 Condition Number

The **condition number** measures isotropy:

$$\kappa = \frac{\sigma_{max}}{\sigma_{min}}$$

- $\kappa = 1$: Isotropic (equal mobility in all directions)
- $\kappa \gg 1$: Anisotropic (much better in some directions)
- $\kappa \to \infty$: At singularity

7.5 Using Manipulability

Applications:

1. **Configuration optimization:** Choose IK solution with highest w
2. **Trajectory planning:** Avoid low-manipulability regions
3. **Workspace design:** Position robot base for high average w
4. **Singularity avoidance:** Monitor w and steer away when low

8 Force-Velocity Duality

8.1 Static Force Mapping

The Jacobian transpose maps end-effector forces to joint torques:

$$\tau = J^T \mathbf{F}$$

Where:

- $\mathbf{F}$: Force/torque at end-effector
- τ: Required joint torques

8.2 Proof via Virtual Work

The principle of virtual work states:

$$\tau^T \delta\mathbf{q} = \mathbf{F}^T \delta\mathbf{x}$$

Since $\delta\mathbf{x} = J\delta\mathbf{q}$:

$$\tau^T \delta\mathbf{q} = \mathbf{F}^T J \delta\mathbf{q}$$

This must hold for all $\delta\mathbf{q}$, so:

$$\tau = J^T \mathbf{F}$$

8.3 Force Ellipsoid

Just as velocity ellipsoid shows motion capability, the **force ellipsoid** shows force capability:

$$\|\tau\| \leq 1 \quad \Rightarrow \quad \mathbf{F}^T J J^T \mathbf{F} \leq 1$$

Interestingly, the force and velocity ellipsoids are **reciprocal**:

- Directions of easy motion → hard to apply force
- Directions of hard motion → easy to apply force

9 Summary

9.1 Key Equations

Equation	Description
$\mathbf{v} = J\dot{\mathbf{q}}$	Velocity kinematics
$\dot{\mathbf{q}} = J^{-1}\mathbf{v}$	Inverse velocity kinematics
$\det(J) = 0$	Singularity condition
$w = \sqrt{\det(JJ^T)}$	Manipulability
$\tau = J^T\mathbf{F}$	Force mapping

9.2 2-Link Arm Summary

$$J = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}$$

$$\det(J) = L_1 L_2 \sin\theta_2$$

Singularities: $\theta_2 = 0, \pm\pi$

9.3 Common Mistakes

1. Forgetting that J depends on configuration
2. Inverting J at or near singularities without damping
3. Confusing J^{-1} with J^T (they're different!)
4. Using numerical Jacobian when analytical is available (less accurate)
5. Ignoring angular velocity for 3D problems

10 Practice Problems

10.1 Problem 1: Jacobian Computation

For a 2-link arm with $L_1 = 1.0$ m, $L_2 = 0.5$ m at configuration $\theta_1 = 45°$, $\theta_2 = 60°$:

a) Compute the Jacobian matrix numerically
b) Compute $\det(J)$ and verify not at singularity
c) If $\dot{\theta}_1 = 0.5$ rad/s and $\dot{\theta}_2 = -0.3$ rad/s, what is $[\dot{x}, \dot{y}]^T$?

10.2 Problem 2: Singularity Analysis

For the same arm:

a) At what configurations is the arm singular?
b) At the singular configuration $\theta_1 = 30°$, $\theta_2 = 0°$, what direction can the end-effector NOT move?
c) What is the manipulability w at $\theta_1 = 30°$, $\theta_2 = 90°$?

10.3 Problem 3: Inverse Velocity

Given desired end-effector velocity $\mathbf{v} = [0.1, 0.2]^T$ m/s at configuration $\theta_1 = 45°$, $\theta_2 = 60°$:

a) Compute the required joint velocities using J^{-1}
b) Verify by computing $J\dot{\mathbf{q}}$
c) What happens if we try this at $\theta_2 = 0$?

10.4 Problem 4: Implementation

Implement a function `track_line(start, end, q0, L1, L2)` that uses resolved-rate control to move the end-effector from `start` to `end` along a straight line.